TELUGU DOCUMENT WRITER ASSISTANT

A Grammarly-like tool for Telugu

Panuganti Bhanu

Pappu Meghana

Patnam Jahnavi

Hemanth Sai Machi Reddy

Data Collection and Corpus

Primary Datasets:

- Telugu Wikipedia Dump (200MB+) → 8,00,000 lines
- Telugu Stories from www.manatelugukathalu.com → 20,000 lines
- Multilingual corpus from kaggle → 34,00,000 lines

Final Dataset:

- Cleaned and merged dataset → 10,36,856 sentences | 9,59,816 unique words

Storage strategy:

- telugu_tokenized_sentences.json → Pre-processed tokens (fast loading: <2s)
- telugu_vocabulary.txt → Complete word index for spell checking
- telugu_ngram_model.pkl → Trained model (~350MB)

Data Preprocessing Pipeline

Unicode-based Tokenization:

- Extract Telugu characters using regex pattern matching.
- Input: "రాజు పుస్తకం చదువుతున్నాడు"
- Output: ['రాజు', 'పుస్తకం', 'చదువుతున్నాడు']

Sentence Augmentation:

- Add boundary markers for n-gram construction.
- Bigrams: ["<s>"] + tokens + ["</s>"].
- Trigrams: ["<s>", "<s>"] + tokens + ["</s>"]

N-gram Construction:

- Build frequency counts for context-target pairs.
- Example: "రామ మరియ నందిని" → {('రామ', 'మరియ'): {'నందిని': 1, ...}}

Frequency Indexing:

- Build unigram counts
- Create prefix groups (2-char) for spell checker
- Generate vocabulary set for fast lookups

Spell Checker - Levenshtein Distance

Edit distance:

- Measures minimum edits (insertions, deletions, substitutions, swaps) to match correct word
- Efficient for single-character typos or phonetic variations

Example word: "రాజు"

Levenshtein distance → {'రాజు': 0, 'రాజి': 1, 'రాజ': 1}

suggest_corrections()

- Compares a given word with vocabulary words in the same prefix group.
- Keeps words with an edit distance ≤ 2.
- Returns top 3 closest suggestions sorted by distance.

check_sentence()

- Splits the input sentence into Telugu words using regex
- For each word, checks if it exists in vocabulary; if not, finds possible corrections.
- Returns a dictionary mapping incorrect words → list of suggestions.

Sentence completion - Ngram Model

Trigram Model with Linear Interpolation

The core prediction engine combines three probability distributions with weighted interpolation

$$P(w_3 \mid w_1, w_2) = 0.9 \times P_{\text{trigram}}(w_3 \mid w_1, w_2) + 0.1 \times P_{\text{unigram}}(w_3)$$

train()

Builds n-gram counts from corpus by iterating through sentences and incrementing context-target pair frequencies

predict_next()

Returns top-K predictions by calculating weighted probabilities: λ_1·ngram_prob + λ_2.unigram_prob

complete_sentence()

Iterative prediction loop that generates up to 20 words by repeatedly calling predict_next() until end-of-sentence marker

Model weights

λ_1 = 0.9 (Trigram), λ_2 = 0.1 (Unigram)

Results & Model Performance

Next word prediction results

- Input: ["రామ", "మరియ"] → Output: నందిని (0.90) | లో (0.01) | రాజు (0.00)
- Input: ["పిల్లల", "పార్క"] → Output: లో (0.91) | రాజు (0.00) | చదువుతున్నాడు (0.00)
- Input: ["బెంగళూర", "లో"] → Output: కొత్త (0.90) | లో (0.01) | రాజు (0.00)

Example input: "సముద్ర"

Predicted output: “సముద్ర మట్టానికి మీటర్ల ఎత్తులో ఉంది”

Performance metrics:

- Vocabulary size: 959816 words
- Training sentences: 1036856
- Perplexity: 7.35

Thankyou